Estructuras de Datos

2026
Primer Cuatrimestre

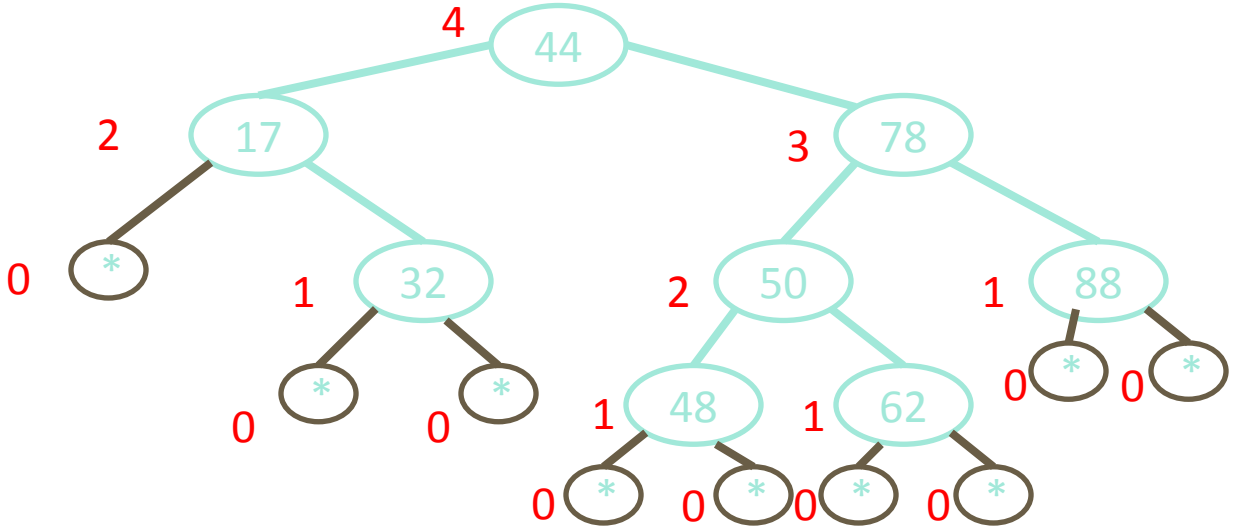
REPASO

Motivaciones

- El árbol binario de búsqueda permite implementar conjuntos y mapeos con un tiempo de operaciones buscar, insertar y eliminar con orden logarítmico en la cantidad de elementos en promedio.
- En el peor caso las operaciones tienen orden lineal en la cantidad de elementos (cuando las inserciones se realizaron en forma ascendente o descendente en cuyo caso el árbol degenera en una lista).
- Hay estructuras alternativas que garantizan tiempo de acceso de orden logarítmico en la cantidad de elementos y se los conoce como árboles de búsqueda balanceados: Árbol AVL, Árbol 2-3 y Árbol B.

Árboles AVL: Ejemplo

- Propiedad del balance de la altura: Para cada nodo interno v de T, las alturas de los hijos difieren en a lo sumo 1.
- Ejemplo: Los * corresponden a nodos nulos (con altura 0 de acuerdo a GT).



Comentarios

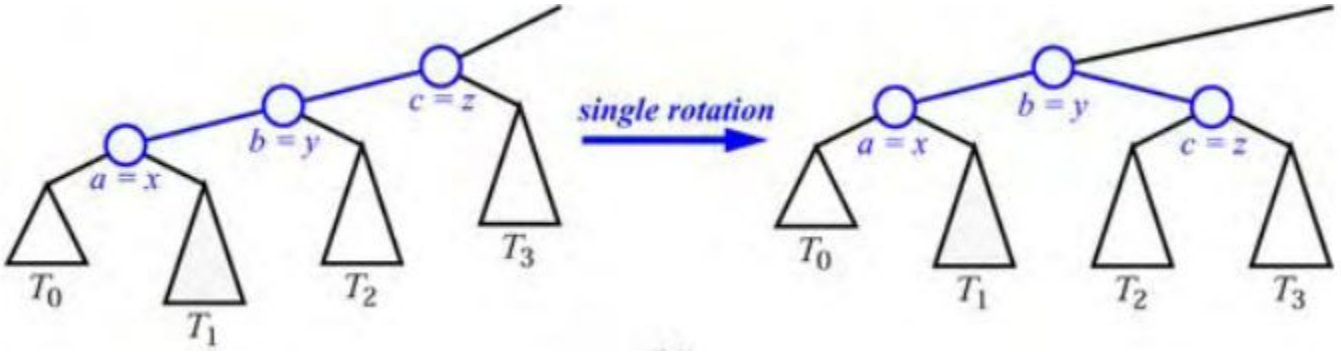
- Siguiendo a Goodrich & Tamassia diremos que los nodos nulos tienen altura 0.
- Como un AVL es un árbol binario de búsqueda, la operación de búsqueda no sufre alteraciones.
- La únicas operaciones a modificar son la de inserción y eliminación, las cuales deben verificar que se cumpla la propiedad de balance al finalizar la operación.
  - Las eliminaciones las haremos perezosas (lazy): marcaremos los datos eliminados con un booleano.
- Luego de cada modificación en un nodo, rebalancearán los hijos de dicho nodo en O(1) por medio de las llamadas “rotaciones”.
- Las modificaciones se hacen desde la hoja donde se insertó el nodo hacia la raíz siguiendo el camino de llamadas recursivas.

Rotaciones

Son cuatro correspondientes a las cuatro combinaciones para la inserción de una clave a partir de un nodo raíz del subárbol considerado:

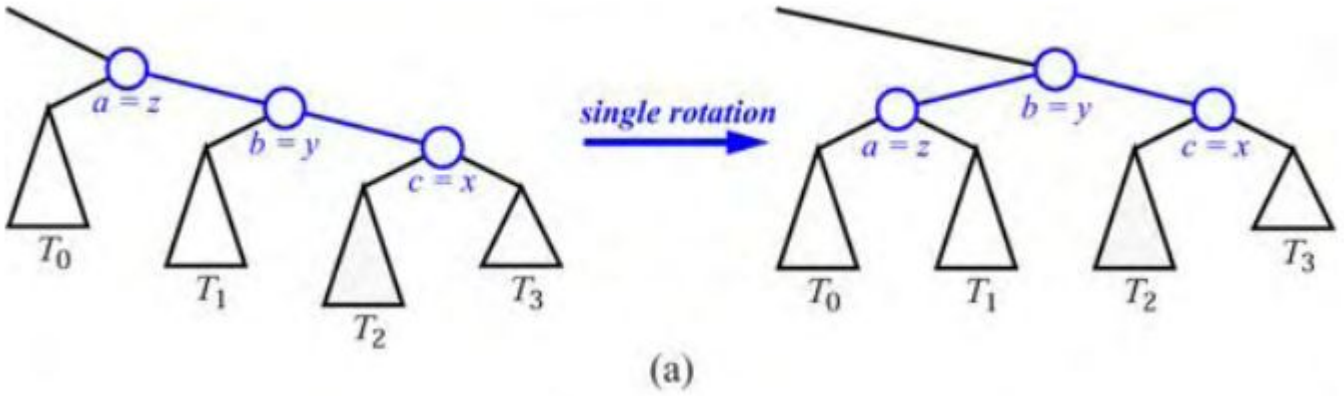
1) izquierda – izquierda: rotación simple de izquierda a derecha
2) izquierda – derecha: Rotación doble de izquierda a derecha
3) derecha – derecha: Rotación simple de derecha a izquierda
4) derecha – izquierda: Rotación doble de derecha a izquierda

(1) Rotación simple de izquierda a derecha (formalización)



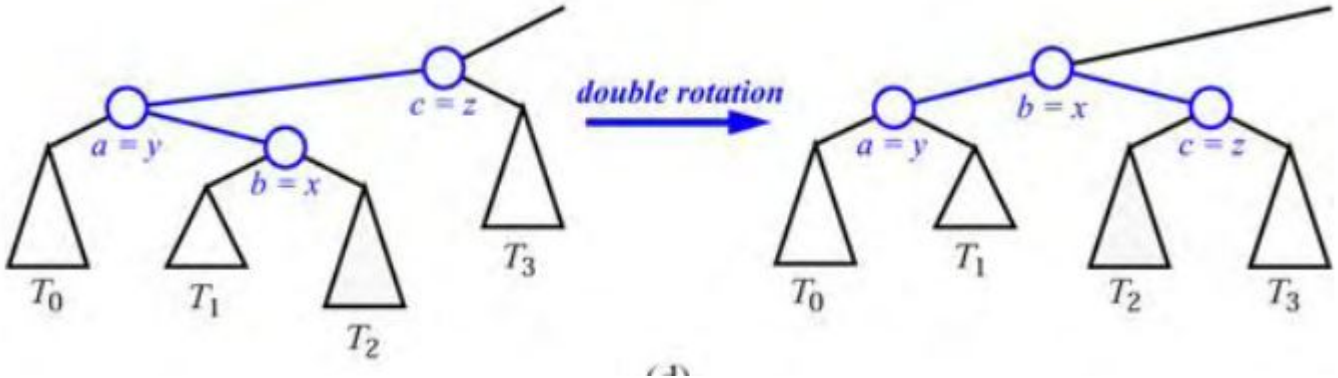
Corresponde a dos inserciones seguidas hacia la izquierda desde la raíz del subárbol considerado

(3) Rotación simple derecha a izquierda: Formalización



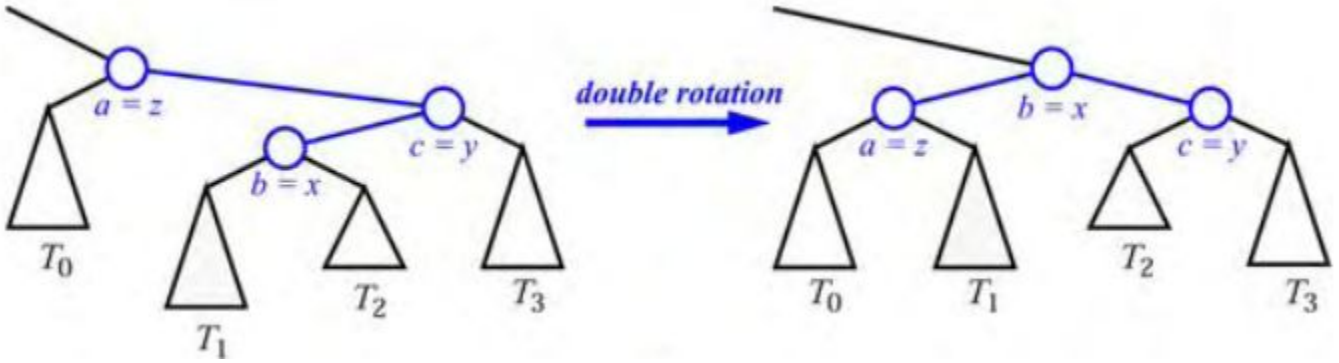
Corresponde a dos inserciones seguidas hacia la derecha de la raíz del subárbol considerado

(2) Rotación izq-der (doble de izquierda a derecha): Formalización



Corresponde a una inserción en el hijo izquierdo y luego en hijo derecho del hijo izquierdo de la raíz del subárbol considerado

(4) Rotación der-izq (doble de derecha a izquierda)



Corresponde a una inserción en el hijo derecho de la raíz y luego en el hijo izquierdo del hijo derechos de la raíz del subárbol considerado

Implementación Java de la Inserción

La única diferencia con el NodoABB es que en cada nodo mantengo la altura de dicho nodo.

```
// Archivo: NodoAVL.java
public class NodoAVL<E>
{
        private NodoAVL<E> padre;
        private E rotulo;
        private int altura;
        private boolean eliminado; // □ diferencia!
        private NodoAVL<E> izq, der;

        public NodoAVL<E>(E rotulo){
                altura = 0;
                // Al crear un nodo dummy anoto que su altura es 0.
                   ...    // Resto Idem NodoABB}
        // setters y getters incluyendo la altura y eliminado
}
```

```
public class AVL<E> El árbol AVL está parametrizado con el tipo E de los rótulos
{
        NodoAVL<E> raiz;                    Su raíz es un nodo AVL
        Comparator<E> comp;                 Necesito el comparador de claves

        public AVL(Comparator<E> comp)      El cliente me pasa el comparador
        {
            raiz = new NodoAVL<E>(null);    Creo un nodo dummy para la raíz
            this.comp = comp;               Salvo el comparador
        }

        public void insert(E x)             Insertar recorre el árbol recursivamente
        {                                   desde la raíz con un método auxiliar
             insertaux( raiz, x );          llamado insertaux
        }

        private int max(int i, int j )      Este método sirve para calcular el
        {                                   máximo entre i y j
             return i>j ? i : j;
        }
```

```
private void insertaux( NodoAVL<E> p, E item ) {  /* p es el nodo actual, la primera vez es la raíz */
  if( p.getRotulo() == null )  {                    // Llegué a un dummy
     p.setRotulo( item );    p.setAltura( 1 );    // Le seteo la altura al dummy como 1 y le hago 2 hijos dummy
     p.setIzq( new NodoAVL<E>( null ) );          p.setDer( new NodoAVL<E>( null ) );
     p.getIzq().setPadre( p );                    p.getDer().setPadre( p );
  }  else {
      int comparacion = comp.compare( item, p.getRotulo() );
      if( comparacion == 0 )  { eliminado=false;   /* ítem ya está en el árbol => no cambia nada */  }
      else if( comparacion < 0 ) {
          insertaraux( p.getLeft(), item );  // Inserto recursivamente en el HI y a la vuelta rebalanceo
          if( Math.abs( p.getLeft().getAltura() - p.getRight().getAltura() ) > 1 ) {
              // Rebalancear mediante rotaciones:  testeo por rotaciones (i) o (ii)
              // Si estoy aca => item < x, debo testear si (item < y) o (item > y)
              // si item < y => rotacion (i);       si item > y => rotacion (ii)
              E x = p.getRotulo();   // no se usa, es solo para la explicación
              E y = p.getLeft().getRotulo();
              E z = p.getLeft().getLeft().getRotulo(); // no se usa, es solo para la explicación
              int comp_item_y = comp.compare( item, y );
              if( comp_item_y < 0 )  rotacion_i(p);   // item < y => rotacion (i)
              else  rotacion_ii(p);  // item > y => rotacion (ii)
          }
     }  else        {
        /* Caso simétrico pero insertando hacia la derecha y luego testeo por rotaciones (iii) y (iv) */
     }
     p.setAltura( max(p.getLeft().getAltura(), p.getRight().getAltura()) + 1 ); // Setear altura del nodo p
 }
}
```

Complejidad temporal de la inserción

- Noten que las rotaciones se hacen en los nodos del camino desde la raíz hasta la hoja donde se insertó la nueva clave.
- Como las rotaciones se implementan con asignaciones de referencias (posiciones), cada rotación se hace en tiempo constante.
- La cantidad de rotaciones es del orden de la altura del árbol.
- La altura es proporcional al logaritmo de la cantidad de nodos del árbol.
- Por lo tanto, el tiempo de insertar es del orden del logaritmo de la cantidad de nodos del árbol.

Árboles 2-3: Definiciones

Un "árbol 2-3" es un árbol tal que cada nodo interno (no hoja) tiene dos o tres hijos, y **todas las hojas están al mismo nivel**.

La definición recursiva es: T es un árbol 2-3 de altura h si:

a) T es vacío (es decir de altura 0)

b) T es de la forma:

donde n es un nodo y T_i y T_d son árboles 2-3 cada uno de altura h-1.
T_i se dice "subárbol izquierdo" y T_d "subárbol derecho".

c) T es de la forma:

donde n es un nodo y T_i, T_m y T_d son árboles 2-3 cada uno de altura h-1.
T_i se dice "subárbol izquierdo“, T_m se dice "subárbol medio“ y T_d "subárbol derecho".

Árboles 2-3: Definiciones

- Propiedad: Si un árbol 2-3 no contiene ningún nodo con 3 hijos entonces su forma corresponde a un árbol binario lleno.

Árboles 2-3: Definiciones

Un arbol 2-3 es un "arbol 2-3 de búsqueda" si T es un árbol 2-3 tal que

a) T es vacío

b) T es de la forma:

n contiene una clave k, y
k es mayor que las claves de T_i
k es menor que las claves de T_d
T_i y T_d son árboles 2-3 de búsqueda

c) T es de la forma:

n contiene dos claves k_1 y k_2, y
k_1 es mayor que las claves de T_i,
k_1 es menor que las claves de T_m
k_2 es mayor que las claves de T_m
k_2 es menor que las claves de T_d
T_i, T_m y T_d son árboles 2-3 de búsqueda

Árboles 2-3: Definiciones

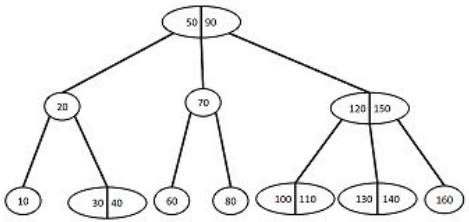
Reglas para ubicar claves en los nodos de un árbol 2-3 de búsqueda:

1) Si n tiene dos hijos, entonces contiene una clave
2) Si n tiene tres hijos, entonces contiene dos claves
3) Si n es una hoja, entonces contiene una o dos claves

Ejemplo de árbol 2-3

Búsqueda en un árbol 2-3

```
Recuperar( R, clave ) : boolean
SI clave está en R ENTONCES
        RETORNAR true
SINO SI R es una hoja ENTONCES
        RETORNAR false
SINO
     SI R tiene una clave k ENTONCES
        SI clave < k ENTONCES  RETORNAR Recuperar( Ti(R), clave )
        SINO RETORNAR Recuperar( Td(R), clave )
 SINO
      SI R tiene dos claves k1 y K2 ENTONCES
        SI clave < k1 ENTONCES
                RETORNAR Recuperar( Ti(R), clave )
        SINO SI clave < k2 ENTONCES
                RETORNAR Recuperar( Tm(R), clave )
                 SINO
                RETORNAR Recuperar( Td(R), clave )
```



Inserción en un árbol 2-3:

- Igual que en el ABB, siempre se inserta en una hoja siguiendo el camino de la búsqueda desde la raíz del árbol hasta la hoja correspondiente (esa hoja ya va a tener por lo menos 1 clave).
  - Si la hoja ahora tiene 2 claves, terminamos.
  - Si la hoja ahora tiene 3 claves, se produce un rebalse (*overflow*) y se debe partir el nodo en 2 nodos, la clave del medio sube al padre, quien queda a cargo de administrar ese hijo que se duplicó y ver dónde ubicar esa clave.
    - Si el padre tenía 1 clave y 2 hijos, no hay problema, porque ahora tendrá 2 claves y 3 hijos.
    - Si el padre ya tenía 2 claves y 3 hijos, ahora pasaría a tener 3 claves y 4 hijos, lo cual no puede ocurrir. Entonces el proceso anterior se repite.
    - El proceso termina cuando terminamos en un nodo intermedio (con 2 claves o 3 hijos) o llegamos a crear una nueva raíz y el árbol crece 1 nivel.
- Como este proceso tiene T(h) = O(h), ocurre que T(n) = O(log(n)).

1) Insertamos 10 en el árbol vacío:

2) Insertamos 15: como hay lugar en la única hoja, allí ponemos la nueva clave y terminamos

3) Insertamos 12: vamos a la única hoja y ponemos el 12 allí, pero hay rebalse porque tenemos 3 claves y sólo tenemos permitidas 2.



Partimos el nodo en 2 nodos dividiendo las claves y le pasamos al padre los 2 nodos junto con la clave del medio. Como no hay padre, el árbol crece en un nivel al crear un nuevo nodo para acomodar la clave con los 2 nodos como sus hijos.

4) Cualquier clave que inserte, siempre va a una hoja siguiendo el criterio de búsqueda. Inserto 20 y 5. Como no hay rebalse porque había lugar en las hojas, terminamos.

5) Inserto 30, el cual termina en el hijo derecho de 12.

Hay rebalse porque tengo 3 claves y solo tengo permitidas 2

Parto el nodo rebalsado en 2 nodos y se los paso a su padre junto con la clave del medio (que es el 20) y terminamos porque la raíz tiene lugar para otra clave y otro hijo extra.

6) Inserto 1 en el árbol del paso (5):

Hay rebalse

Parto el nodo y subo el 5 al padre (que es la raíz). Pero ahora la raíz tiene rebalse (3 claves y 4 hijos, situación no permitida).

Parto el nodo raíz en 2 nodos y subo el 12 creando una nueva raíz

RESUMEN: Para insertar un valor X en un árbol 2-3,
1) hay que ubicar la hoja L en la cual X terminará.
1.1) Si L contiene ahora dos claves, terminamos.
1.2)Si L contiene tres claves, hay que partirla en dos hojas L1 y L2. L1 se queda con la clave más pequeña, L2 con la más grande, y la del medio se manda al padre P de L. Los nodos L1 y L2 se convierten en los hijos de P.
1.2.1) Si P tiene sólo 3 hijos (y 2 claves), terminamos.
1.2.2) En cambio, si P tiene 4 hijos (y 3 claves), hay que partir a P igual que como hicimos con una hoja sólo que hay que ocuparse de sus 4 hijos.
1.2.2.1) Partimos a P en P1 y P2, a P1 le damos la clave más pequeña y los dos hijos de la izquierda y a P2 le damos la clave más grande y los dos hijos de la derecha.
1.2.2.2) La clave que sobra se manda al padre de P en forma recursiva.
1.2.2.3) El proceso termina cuando la clave sobrante termina en un nodo con dos claves o el árbol crece 1 en altura (al crear una nueva raíz).

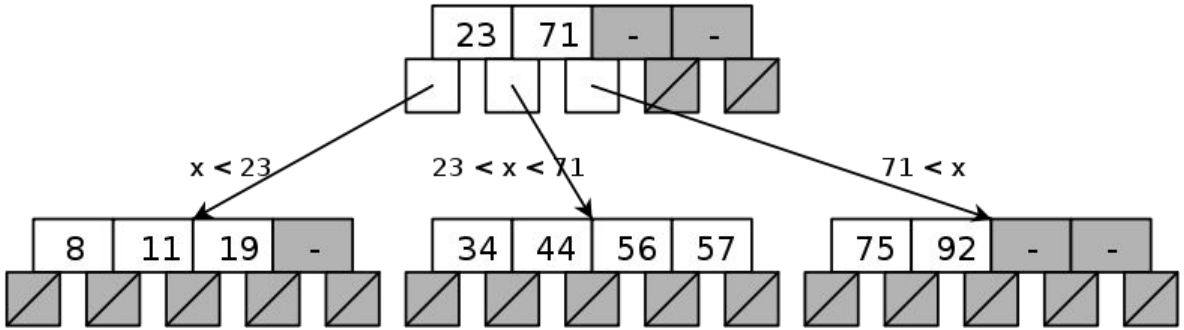
Árbol B

- Un árbol-B (B-tree) es un árbol balanceado (o auto-balanceante) que mantiene los datos ordenados y permite realizar búsquedas, inserciones y eliminaciones en tiempo logarítmico.
- Se puede ver también como una generalización del árbol 2-3 porque sus nodos pueden tener más de tres hijos.
- A diferencia de los árboles 2-3, el árbol-B está optimizado para sistemas que leen y escriben grandes bloques de datos.
- Los árboles-B son un buen ejemplo de una estructura de datos para memoria externa (en disco) y se usan comúnmente en bases de datos y sistemas de archivos.

Arboles B (o M-arios de búsqueda)

- Los árboles B se optimizan para manejar grandes volúmenes de datos.
- Los árboles B se almacenan en disco y el tamaño del nodo coincide con un múltiplo entero del tamaño del sector del disco.
- Se utilizan para implementar índices en bases de datos.
- El grado de ramificación d del árbol es un entero, indicando que un nodo tiene entre d y 2d claves y entre d+1 y 2d+1 hijos (excepto la raíz que puede tener menos de d claves y tiene por lo menos 1 clave y 2 hijos).
- Cuando d=1 tenemos un árbol 2-3.
- Dependiendo de la formalización, al número 2d+1 se lo llama M (i.e. tenemos M-1 claves y M hijos a lo sumo por nodo; en un árbol 2-3, M vale 3)

Árbol B



- El tiempo de búsqueda, inserción y eliminación es en tiempo logarítmico puesto que es del orden de la altura del árbol.
- Si d=1000, cada nodo tiene d claves y d+1 hijos por lo menos, entonces la altura del árbol es del orden $\log_d(n)$ con n = cantidad de claves del árbol.
  - Ej: si d=1.000 y n=1.000.000, entonces $\log_d(1.000.000)=2$ entonces son 3 lecturas (como la altura es 2, el camino tiene 3 nodos).
- Generalmente hay espacio desperdiciado en los nodos (i.e. fragmentación interna).

Árboles B: Inserción

- Comenzamos con un nodo vacío (el cual funciona como un arreglo ordenado).
- Las claves se insertan (en forma lineal y ordenada) en el nodo hasta tener a lo sumo 2d = m-1 claves.
- Luego, al insertar la siguiente clave, el nodo rebalsa.
- El nodo se parte en dos (cada uno con d claves): la clave del medio va al nodo de arriba (incrementando la altura del árbol cuando tal nodo de arriba no existiera) y los nodos que quedan se quedan cada uno con la mitad de las claves menores a d y mayores a d respectivamente.

Supongamos que d=3, entonces tendremos entre d=3 y 2d=6 claves por nodo excepto la raíz que puede tener menos claves (es decir, M=2d+1=7, el B-árbol es de orden 7) y siempre el número de hijos de un nodo es uno más que su cantidad de claves.
Insertemos las claves 1, 40, 5, 3, 45, 8, 9.

Inserto 1:

Inserto 40:

Inserto 5 (hago shift):

Inserto 3 (hago shift):

Inserto 45:

Inserto 8:

La próxima inserción, es decir la del 9, producirá un rebalse porque el nodo ya está lleno.

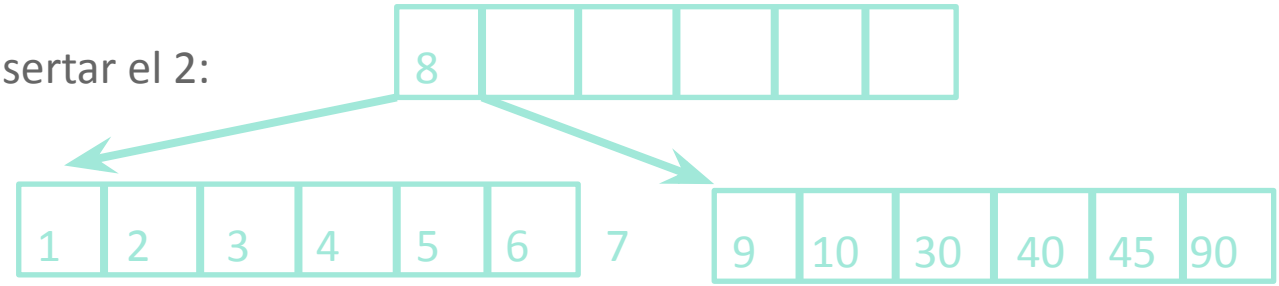
Estado antes de insertar 9:

Entonces partimos el nodo L en L1 y L2 y creamos un nodo más arriba con L1 y L2 como hijos y como clave la clave del medio de L considerando también al 9, que en este caso será el 8:

Las inserciones se hacen siempre en las hojas siguiendo el orden de las claves.
Para insertar una clave, habrá que ver si es menor o mayor a la clave de la raíz.
Insertar 10, 4, 30, 7, 6, 90 produce el árbol:

¡Insertar el 2 producirá un rebalse en la hoja de la izquierda!

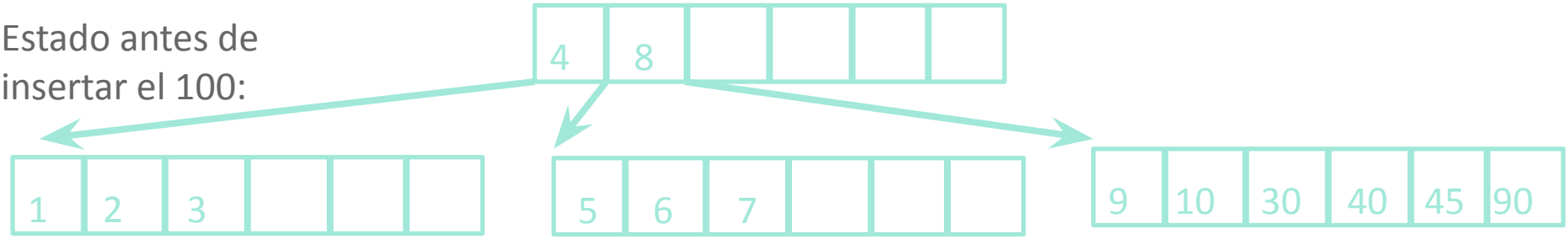
Estado al insertar el 2:



Rebalsó el 7, vamos a partir nodo reabalsado en dos y mandar la clave del medio (es este caso 4) al padre:

Insertar 100 producirá un rebalse en la hoja de más a la derecha.
El espacio desperdiciado en los arreglos constituye la *fragmentación interna (espacio de memoria desperdiciado dentro de las estructuras de datos)*.

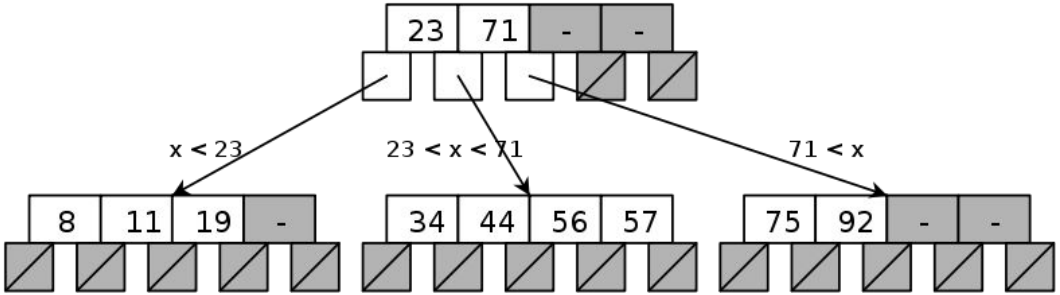
Estado antes de insertar el 100:



Al insertar el 100, como 100 > 8, va hacia la hoja de más a la derecha, que rebalsa. Entonces la partimos y subimos la clave del medio (en este caso el 40):

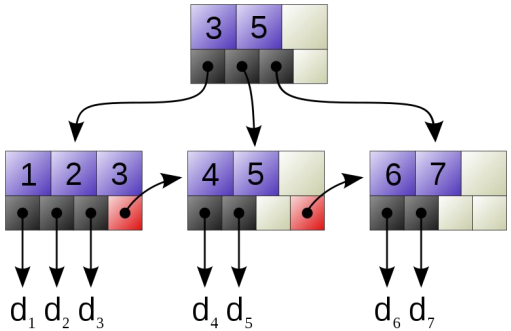
Este proceso continúa hasta que los rebalses sucesivos en las hojas van llenando la raíz del árbol. En el siguiente rebalse, se partirá la raíz en 2 y se crea una nueva raíz haciendo crecer el árbol en un nivel.

Árboles B: Búsqueda de k



- Empezamos a buscar a partir del nodo p=raíz.
- CB: Si k está en p => termino con éxito.
- CB: Si k es una hoja y k no está en p => fallamos.
- CR: Si k no está en p, buscamos k en el hijo entre las dos claves k_i y k_{i+1} de p tales que $k_i \leq k \leq k_{i+1}$.

Variante del árbol B: Árbol B+



- El árbol B+ almacena entradas (clave,valor) sólo en las hojas, los nodos internos almacenan solo claves que sirven para guiar la búsqueda en O(h).
- Recorrer las hojas de izquierda a derecha iterativamente permite listar las claves en forma ascendente en O(n).
- Ejemplo: 3 en la raíz indica la mayor clave del primer hijo del siguiente nivel.

5 en la raíz indica la mayor clave del segundo hijo del siguiente nivel.

d_1, d_2, d_3, d_5, d_6, d_7 corresponden a los valores de las entradas para las claves 1,2,3,4,5,6,7 (punteros al archivo de datos).

Bibliografía

- Árboles AVL: Capítulo 10, Secciones 2 y 4 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.
- Árboles 2-3: Basado en Paul Helman & Robert Veroff. Intermediate Problem Solving and Data Structures. Walls and Mirrors, Benjamming Cummings, Menlo Park, 1986.
- Árboles B: Capítulo 14, Sección 3 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.
- Justificación performance AVL:

http://www.openbookproject.net/books/pythonds/Trees/AVLTreePerformance.html

http://www.geekyarticles.com/2017/04/fibonacci-sequence-and-worst-avl-trees.html?m=1

https://es.wikipedia.org/wiki/Número_áureo